The Origin Story (What & Why)

Go (Golang) is an open-source, compiled, statically typed programming language designed by Google.

It is built for Simplicity, Reliability, and Efficiency.

The Name: It is officially called "Go." The term "Golang" comes from its website domain (golang.org) because "go.org" wasn't available.
You can use them interchangeably, but "Go" is the proper name.

"Compiled": Unlike Python or JavaScript which are interpreted (read line-by-line),
Go is translated directly into machine code (0s and 1s) that the computer understands.
This makes it very fast.

"Statically Typed": This means the computer checks your code for errors before running it.
If you try to do math with text, Go stops you before you even start.

The "Why" (The Problem)

The 2007 Landscape

C++ / Java: Fast execution, but complex and very slow to compile.

Python / Ruby: Easy to write and slow execution, but fast to test.

The Google Problem: Builds were taking 45 minutes. Developers were waiting, not coding.

Go was conceived in 2007 at Google.

The Creators: It was created by three legends:

Ken Thompson (Inventor of UNIX and the B language).

Rob Pike (UNIX team, UTF-8).

Robert Griesemer (Java HotSpot VM).

Legend has it that they were waiting for a massive C++ project to compile (which took 45 minutes).
During that wait, they decided to design a new language.

The Trade-off: Before Go, you had to choose:

Efficient execution (C++) OR

Easy development (Python).

You couldn't have both. Go was designed to bridge this gap.

The Solution (Go's Superpowers)

Simplicity: Minimal keywords (only 25!).

Concurrency: Built for multi-core processors.

Fast Compilation: Builds in seconds, not minutes.

Batteries Included: Powerful Standard Library.

Simplicity: Go is boring, and that is a compliment. It refuses to add complex features
(like Generics for a long time, or inheritance).
This means a new developer can read an expert's code and understand it in a day.

Concurrency: Most languages were designed when computers had 1 CPU core.

Go was born in the era of Multi-core processors and Cloud Computing.

It uses Goroutines (lightweight threads) to run thousands of tasks at the same time without crashing the system.

Performance: It runs almost as fast as C or C++, but it feels as easy to type as Python.

Who Uses It?

Cloud Infrastructure: Docker, Kubernetes, Terraform (All written in Go).

Companies: Google, Netflix, Uber, Dropbox, Twitch.

"If you use the internet today, you are using Go."

Go has become the "Language of the Cloud." Almost all modern cloud tools
(like Docker and Kubernetes) are written in Go because of its speed and
ability to handle network traffic efficiently.

Installing and Verifying Go

Before writing code, always verify Go is installed.

```
go version
```

Example output:

```
go version go1.20 linux/amd64
```

Creating Your First Go Project (Module)

What is a Go Module?

- A Module is a collection of Go packages stored together.
- It is defined by a file called go.mod.
- It handles versioning and dependencies.

The "Box" Analogy: Imagine you are building a Lego castle. The Module is the baseplate that holds all the pieces together. Without the baseplate, your pieces (files) would just be a messy pile.

Why do we need it? In the early days of Go, you had to put all your code in one specific folder on your computer (called the GOPATH). It was a nightmare. Modules allow you to create your project anywhere on your computer.

Dependency Management: If your project needs to use a tool someone else wrote (like a database driver), the Module tracks exactly which version you are using so your code doesn't break when they update theirs.

Step 1: Create a project directory

```
mkdir ch1
cd ch1
```

We create a physical folder. This is the root of your project.

Step 2: Initialize a Go module

Crucial: You must run the initialization command inside the folder you want to turn into a module.

```
go mod init <module-name>
```

Example:

```
go mod init hello_world
go mod init example.com/hello
```

What just happened?

Go just created a text file named go.mod.

Anatomy of the go.mod File

`go.mod`

```
module hello_world
go 1.25.1
```

`module <name>`: This identifies your project.
Other files in your project will use this name to find each other.
Name of the project (used when importing)

`go 1.xx`: This is the minimum version of Go required to run this code. It acts as a safety guard.

Important rule: Never manually edit go.mod
Use Go commands instead:

Writing Your First Go Program

The File Creation

Extension: All Go source code files end in .go.

Naming: We usually call the entry file main.go, but you can name it hello.go if you prefer.

Code:

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

Understanding a Go Program (Line by Line)

`package main`

The "Identity" of the file.

The Rule: Every single Go file must start with a package <name> declaration.
It tells the compiler which group this file belongs to.

Why main? This is special.

- If you name it package calculator, Go thinks you are building a Library (a tool for other programs to use).
- If you name it package main, Go knows this is an Executable (a program that can run on its own).

Analogy: It's like putting a label on a box. package main labels this box as "The Engine."

```go
import "fmt"
```

Loading the Toolkit.

What is it? We need to print text to the screen, but the base Go language doesn't
know how to talk to your screen. We need a tool for that.

fmt: Stands for "Format" (pronounced fumpt).
It is a package inside Go's Standard Library (the tools that come installed with Go).

The Strings: Notice fmt is inside quotes " ". This is the import path.

```go
func main()
```

The Entry Point

main: When you run your program, Go looks for a function specifically named main to start the execution.

- Program execution starts here
- No arguments
- No return value
- Exactly one main function allowed

```go
fmt.Println("Hello, world!")
```

Package.Function. We are calling the Println function inside the fmt package.

Println: Stands for "Print Line." It prints the text and then hits "Enter" (adds a new line) at the end.

Building & Running Your Program

```
go run main.go
```

It compiles your code into a temporary folder, executes it, and then deletes the temporary file.

When to use it: During development. When you are making changes every 5 seconds and want to see the result instantly.

```
go build
```

It looks at your go.mod file, finds the module name, and creates a permanent, executable Binary file.

The Binary: After running this, look in your folder. You will see a new file
(on Windows, it ends in .exe; on Mac/Linux, it has no extension).

Portability: You can send this file to a friend who doesn't even have Go installed, and it will run.
This is one of Go's greatest strengths: Static Binaries.
Everything the program needs is packed inside that one file.

Execution: You run it by typing ./hello_world (or whatever your module is named).

```
go build -o <name>
```

Command: go build -o super-app

The -o Flag: Stands for Output.

Result: Instead of naming the file after your module, Go will name the executable super-app.

Cross-Compilation

You can build a Windows .exe while using a Mac, or a Linux binary while using Windows.

Commands (Demo only):

```
To build for Windows: GOOS=windows go build

To build for Mac: GOOS=darwin go build

To build for Linux: GOOS=linux go build
```

In other languages, this is very hard. In Go, it's just one command.
This is why Go is the king of Cloud tools—developers write the code
on their laptops and build it for the Linux servers where the code will actually live.

Go Formatting & The Semicolon Rule

Standardization: Go was designed for large teams at Google. To make code readable by everyone,
the creators made a tool that automatically formats your code.

Tabs vs. Spaces: Go uses Tabs for indentation. If you use spaces, go fmt will change them back to tabs.

The Benefit: You never have to "pretty-print" your code manually.
You just write messy code, run the tool, and it becomes perfect.

Running formatter

```
go fmt ./...
```

./... → current directory + all subdirectories

Formats all Go files

The Semicolon Insertion Rule

Like C or Java, Go requires semicolons at the end of statements.

However, you (the human) never type them.

The Go Lexer (the part of the compiler that reads your code) inserts them automatically based on specific rules.

How it works: When the compiler reads your code, it looks at the very last "token" (word or symbol) on a line.

If that line ends in something that looks like the end of a thought (like a closing parenthesis),
a return statement, or a variable name), the compiler secretly adds a ; right there.

- Identifier (x, int, float64)
- Literal ("text", 123)
- return, break, continue
- ++, --
-), }

Why it matters:

- Makes compiler simpler
- Improves performance
- Enforces consistency

Why brace placement matters

```
func main()
{
        fmt.Println("Hello")
}
```

Compiler sees:

```
func main();
{
        ...
};
```

The Correct Way:

```
func main() { // No semicolon added here because it ends in a brace!
    fmt.Println("Hello")
}
```

Detecting Sneaky Bugs (go vet)

Command: go vet ./...

What it does: It looks for code that is technically legal (it will compile) but is almost certainly a mistake.

Example

```
// This will compile, but it's wrong!
fmt.Printf("Hello %s")
```

Explanation: Printf expects a second argument to fill in the %s.

If you run go build, it works.

If you run go vet, it screams: "Printf format %s reads arg #1, but call has 0 args."

Automation (Makefiles)

Create a file named Makefile (no extension).

Content:

```
.DEFAULT_GOAL := build

.PHONY: fmt vet build

fmt:
    go fmt ./...

vet: fmt
    go vet ./...

build: vet
    go build
```

The Problem: Running go fmt, then go vet, then go build every time is tedious.
If you forget a step, you might ship bad code.

The Solution: A Makefile. This is a script that has been used in Unix since 1976 to automate tasks.

How it works:

Targets: The words before the colon (e.g., build:, fmt:). These are the commands you can run (e.g., make build).

Dependencies: build: vet means "Before you run the build step, you MUST run the vet step."

Chaining: vet: fmt means "Before you vet, you must format."

The Result:

Running just one command (make) will now:

1. Format the code.
2. Check for errors (vet).
3. Compile the binary.